Theater Ticketing System

Software Requirements Specification

0.6

3/12/26

Group 4
James Tuazon, Kazuki Thorne

Prepared for
CS 250- Introduction to Software Systems
Instructor: Gus Hanna, Ph.D.
Spring 2026

Revision History

Date	Description	Author	Comments
2/12/26	0.1 - Initial edits/notes	K. Thorne, J. Tuazon	Rough notes for a few sections, initial labels set
2/17/26	0.2 - Section 2 and 3 notes	K. Thorne, J. Tuazon	Rough notes for sections 2 and 3
2/19/26	0.3 Drafting for new classes	K. Thorne, J. Tuazon	Preparation for first submission
3/4/26	0.4 Drafting for UML diagram	K. Thorne, J. Tuazon	Rough Notes for diagrams
3/5/26	0.5 UML diagram and class description finalization	K. Thorne, J. Tuazon	Finalizing diagrams and descriptions
3/12/26	0.6 Test plan draft	K. Thorne, J. Tuazon	Rough draft for testing plan and test cases
3/19/26	0.7 Finishing touches of planning	K. Thorne, J. Tuazon	Finalizing test cases and plan

Document Approval

The following Software Requirements Specification has been accepted and approved by the following:

Signature	Printed Name	Title	Date
	James Tuazon, Kazuki Thorne	Software Eng.	2/19/26
	Dr. Gus Hanna	Instructor, CS 250	

Table of Contents

Revision History ii

Document Approval ii

1. Introduction 1

1.1 Purpose 1
1.2 Scope 1
1.3 Definitions, Acronyms, and Abbreviations 1
1.4 References 1
1.5 Overview 1

2. General Description 2

2.1 Product Perspective 2
2.2 Product Functions 2
2.3 User Characteristics 2
2.4 General Constraints 2
2.5 Assumptions and Dependencies 2

3. Specific Requirements 2

3.1 External Interface Requirements 3
3.1.1 User Interfaces 3
3.1.2 Hardware Interfaces 3
3.1.3 Software Interfaces 3
3.1.4 Communications Interfaces 3
3.2 Functional Requirements 3
3.2.1 <Functional Requirement or Feature #1> 3
3.2.2 <Functional Requirement or Feature #2> 3
3.3 Use Cases 3
3.3.1 Use Case #1 3
3.3.2 Use Case #2 3
3.4 Classes / Objects 3
3.4.1 <Class / Object #1> 3
3.4.2 <Class / Object #2> 3
3.5 Non-Functional Requirements 4
3.5.1 Performance 4
3.5.2 Reliability 4
3.5.3 Availability 4
3.5.4 Security 4
3.5.5 Maintainability 4
3.5.6 Portability 4
3.6 Inverse Requirements 4
3.7 Design Constraints 4
3.8 Logical Database Requirements 4
3.9 Other Requirements 4

4. Design Models 4

4.1 System Description 5
4.2 Software Architecture Diagram 5
4.2.1 Software Architecture Description 5
4.2.1.1 Component Overview 5
4.2.1.2 Flow 5
4.3 UML Diagram 5
4.3.1 UML Description 5

5. Development Plan and timeline 5

5.1 Development Plan 5
5.2 Timeline 5
6. Testing 6
6.1 Testing Plan
6.1.1 Unit Level Testing
6.1.2 Integration Level Testing
6.1.3 System Level Testing
6.2 Test Cases
A. Appendices 6
A.1 Appendix 1 6
A.2 Appendix 2 6

Introduction

1.1 Purpose

The purpose of this document is to collect and analyze ideas gathered for a Theater Ticketing System. It will cover the system's features, design, and constraints in which the system must operate. This document is intended for the developers and clients who have a stake in the system's development.

1.2 Scope

This software system will be a web browser-based Theater Ticketing System for a local San Diego theater chain. The system is designed for the theater to sell tickets to users, where they can either purchase them online via a web browser or in person via a kiosk. The system should include an administrative mode for theater employees to handle errors and change showtimes. Specifically, the system should be able to handle at least 1000 users, block bots that try to purchase large quantities of tickets, include a database of showtimes, movie selections, theater numbers, and available seats, a 20-ticket purchase limit, the ability to purchase tickets two weeks prior up until 10 minutes after showtime, the ability to to submit customer feedback, support different discounts, and display online reviews and critic quotes of movies.

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition

SRS	A Software Requirements Specification is a document that describes all the functions proposed for a system and the constraints under which it must operate. For example, this document.
TTS	Theater Ticketing System
Kiosk	A physical terminal inside the theater for customers to order tickets in person.

1.4 References

Title	Document Number	Version/Date	Publishing Organization	Source
IEEE Recommended Practice for Software Requirements Specifications	IEEE 830-1998	October 20, 1998	IEEE	https://standards.ieee.org/ieee/830/1222/

1.5 Overview

The next section, Section 2, will provide a general description of the system. This section should include how the product will work in relation to other products, a summary of the product's functions, characteristics of intended users, design constraints, and changes that can affect requirements. Section 3 will cover specific requirements, used to guide software design, implementation, and testing. This section should include the technical specification, such as the functional and non-functional requirements of the product.

2. General Description

2.1 Product Perspective

The Theater Ticketing System (TTS) is an independent system consisting of an online website and an in-theater kiosk to order tickets. The system interacts with an external payment gateway to handle transactions. The system also interacts with a printer to print in-person receipts and tickets, as well as an email account system and text messaging to email customers digital tickets

and receipts. The system has access to a database of critic reviews and data to pull from and display. This system aims to replace the current theater ticketing system by adding new features such as an administrative mode, applying discounts, and viewing critics' reviews.

2.2 Product Functions

Browse Movies: Customers can filter movies by genre, view showtimes, and access external critic reviews/ratings.

Add Discounts: Customers can add discounts such as senior, student, military, etc.

Ticket Type: Customers should be able to select regular versus deluxe tickets (Reserved seating).

Payment Processing: Customers should be able to pay using credit/debit through the system via a payment gateway.

Ticket Delivery: Customers can choose how to receive tickets, a code via email/text, online, or at a physical terminal to get them printed in person.

Accessibility Features: Changing language, font size. Supports English, Spanish, and Swedish.

2.3 User Characteristics

General User: Expected to have varying degrees of technical skill, but mostly basic digital familiarity, the interface must be intuitive enough so that an inexperienced user using it for the first time will be able to navigate the system.

Theater Staff: Expected to have a higher level of technical skill to navigate and manage the system, as well as confront any issues with the system. They will have access to alter movie show times and handle refunds/special discounts.

2.4 General Constraints

The system must be compatible with web browsers, such as Chrome and Firefox.

The system will only be available via a web browser, not a mobile app.

The kiosk will utilize a touch screen and have a limited screen size and hardware.

Kiosk will implement a simplified navigation menu compared to the desktop version. Both the web browser and kiosk versions will display similar information.

2.5 Assumptions and Dependencies

Assumptions:

Assumed that the theater has a stable internet connection to run administrative functions and sync kiosk information with web browser information.

Assumed that kiosk hardware is available, printers, and card scanners.

Assumed that the theater staff would be trained in administrative functions.

Dependencies:

The system depends on third-party credit transaction software.

The system depends on a connection to review sites such as Rotten Tomatoes to display ratings and critic quotes.

3. Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interfaces

The website's UI will have menus, links, displays, and a search function that the user will be able to navigate using a keyboard and mouse.

The kiosk will have a simplified UI in comparison to the web version. Will maintain a similar layout and information. Will utilize a touch screen and a digital keyboard for the search function.

3.1.2 Hardware Interfaces

For a kiosk, the system must have a card reader connected to a third-party payment system and be able to print receipts.
The kiosk must be able to interact with a printer (either an ink or a heat-based printer) to print unique tickets.

3.1.3 Software Interfaces

The system must utilize a third-party payment gateway to handle transactions. A message will be shown to the user if the transaction was either successful or failed. The system must also have access to an email account and a texting account to send tickets and receipts.

3.1.4 Communications Interfaces

The kiosk application and website must be able to communicate with the theater system server to have access to the same information.
The system must also have access to a database of available seats and movies.

3.2 Functional Requirements

3.2.1 Functional Requirement #1 Purchase Tickets

3.2.1.1 Introduction
The application shall allow users to browse or search for a movie, select a showing time, reserve a seat if applicable, and purchase a ticket.

3.2.1.2 Inputs

The user must select a movie and a showtime. Input how many tickets they want. Reserve a seat if the tickets are deluxe through a UI displaying theater seats. User pays for tickets either through a card reader or by entering card information.

3.2.1.3 Processing
The system will check if seats are available.
If deluxe, the system will reserve seats for the user.
After the system verifies the order of tickets, the user pays utilizing a third-party payment system.

3.2.1.4 Outputs
A visual with confirmation text and a summary of the order will be displayed. If a web browser is used, tickets will either be texted or emailed along with a receipt. Kiosks will immediately print the tickets with a receipt.

3.2.1.5 Error Handling
If a seat is taken during the reservation process, the system will display an error message “Seat is not available.”

If a purchasing error occurs, such as a timeout, insufficient funds, or a connection lost, an error message will be displayed, and an option to contact an administrator will appear.

3.2.2 Functional Requirement #2 Idle Kiosk

3.2.2.1 Introduction
Reset the Kiosk to the default when not in use.

3.2.2.2 Inputs
Lack of input for 120 seconds.
After 90 seconds of inactivity, the system will prompt the user to interact with the kiosk, or else cancel the transaction.

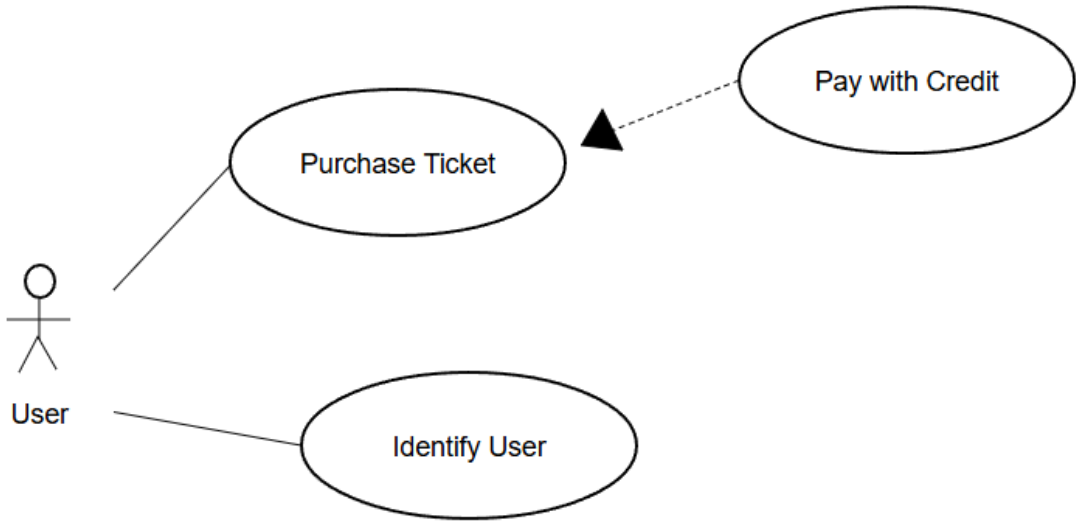
3.2.2.3 Processing
The system will clear any progress that was made during the period of use.

3.2.2.4 Outputs
The screen will return to a default screen that cycles through movie covers.

3.2.2.5 Error Handling
If an error occurs on the kiosk, a call button to call the theater staff is available, and the kiosk will display a decommissioned message.

3.3 Use Cases

3.3.1 Use Case #1 Purchase Ticket Online

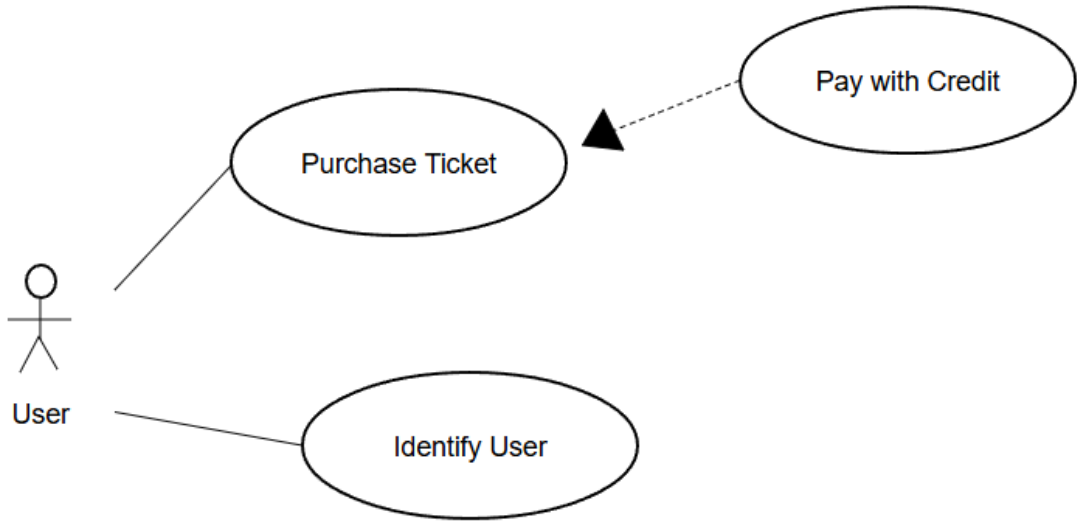


Actor: Online Customer
Precondition: The site front page is displayed with a catalog of movies to browse.
Basic Path:

1. The user finds a movie to watch either by browsing the catalog or by searching for it using the search function.
2. The user has the option to purchase as a guest or log in
3. The user selects the showing time of the movie they want to watch
4. The user inputs how many tickets they want to purchase (max of 20).
5. The system displays ticket prices.
6. User selects checkout and inputs credit/debit card information, and email
7. System verifies purchase.

Post-Condition: System emails receipt and digital tickets and prompts the user if they want to print their tickets

3.3.2 Use Case #2 Purchase Ticket with Kiosk

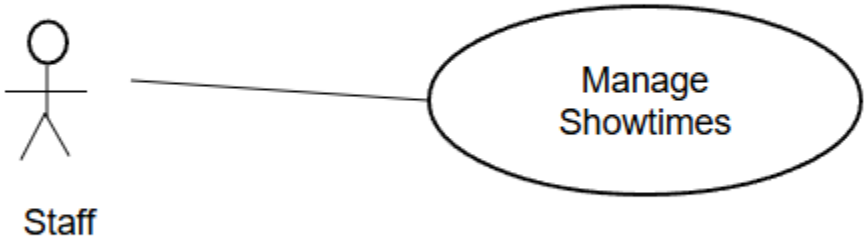


Actor: Kiosk Customer
Precondition: Kiosk displays the default screen.
Basic Flow:

1. Customer touches the screen to start.
2. The user has the option to purchase as a guest or log in
3. Customer browses movie catalog or uses search function.
4. The customer selects the showing time of the movie they want to watch.
5. Customer inputs how many tickets they want to purchase (max of 20).
6. The system displays the price
7. User selects checkout and inserts/taps card onto the terminal
8. The system asks the customer for their phone number to text digital tickets, or if they want it printed
9. System verifies purchase.

Post-Condition: Kisok prints tickets or Texts digital tickets

3.3.3 Use Case #3 Admin Change Showtimes



Actor: Kiosk Theater Staff
Precondition: Showtimes of the movie are not displayed / incorrect.
Basic Flow:

1. Staff selects the movie they want to edit.
2. Adds or Edits showtimes for the movie
3. System prompts for confirmation for changed
4. Staff confirms change

Post-Condition: Showtimes of specific movie are displayed.

3.4 Classes / Objects

3.4.1 Class Object 1 Movie

Movie
showtimes: time, isDeluxe: bool movieName: string ticketPrice: double ratings: double movieDescription: string seatAvailable: bool
setAvailability() : void updatePrice() : void reserveSeat() : void

3.4.2 <Class / Object #2> <Ticket>

Ticket
ticketID: int seatNumber: int price: double isScanned: bool
validate(ticketID) : void generateCode() : int

UserPortal
username: String password: String name: String email: String
logIn(): void

EmployeePortal
employeeUser: String employeePassword: String
logIn(employeeUser, employeePassword): void updateTime(): void updateMovies(): void reserveSeat(): void cancelSeat(): void

PurchaseSeat

cardNumber: int cvv: int expiration: int
getCardInfo(): cardNumber, ccv, expiration passCardInfo(): void validatePayment(): void

3.5 Non-Functional Requirements

3.5.1 Performance

The system shall handle at least 1000 concurrent users up to 10 million.
The system shall load processes within 2 seconds.

3.5.2 Reliability

90% of users using the system for the first time should be able to purchase tickets without assistance.
The website should be up 99.9% of the time.

3.5.3 Availability

The movie should be available on the date and showtime.
The kiosk should be available during the theater's operating hours.
The system should provide language support for English, Spanish, and Swedish
The system should have a light and dark mode

3.5.4 Security

The system shall block bots trying to purchase a large number of tickets for high-demand movies.
Tickets must be unique and non-replicable.
The system keeps daily logs of ticket purchases

3.5.5 Maintainability

The system should have regular updates and an administrator available during operating hours to handle errors.

3.5.6 Portability

The system should be available on most major web browsers, such as Chrome and Firefox.

3.6 Inverse Requirements

*State any *useful* inverse requirements.*

3.7 Design Constraints

Specify design constrains imposed by other standards, company policies, hardware limitation, etc. that will impact this software project.

3.8 Logical Database Requirements

Will a database be used? If so, what logical requirements exist for data formats, storage capabilities, data retention, data integrity, etc.

3.9 Other Requirements

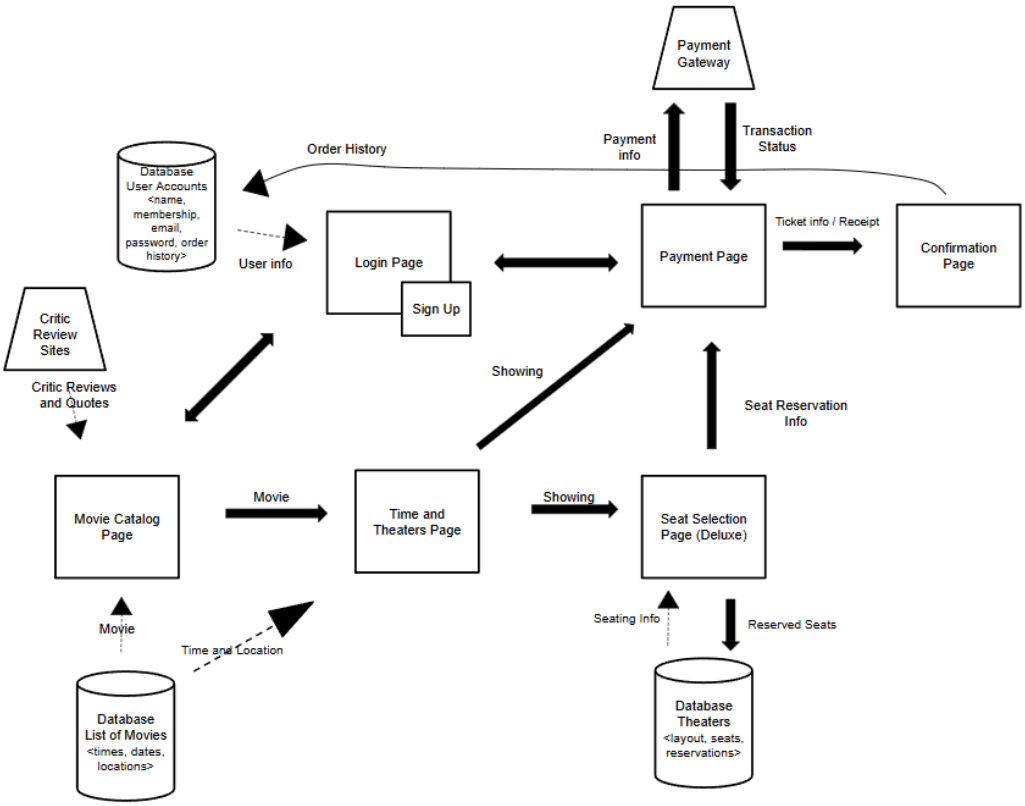
Catchall section for any additional requirements.

4. Design Models

4.1 System Description

The Theater Ticketing System is a web-based ticketing system that allows users to buy tickets and reserve seats from either a web browser or an in-person kiosk in theaters. The system is built using three interconnected components. The user interface, the frontend, is the visual part of the system; this is the part the user sees and interacts with. On the website, the user inputs via keyboard and mouse, and in person via a touchscreen kiosk. The backend has three databases where all the information is stored and pulled from to be displayed on the system. The last component the system utilizes is third-party services. The third-party services used for this system are critic review sites to pull reviews and quotes to display, and a payment gateway to handle transactions.

4.2 Software Architecture Diagram



4.2.1 Software Architecture Description

4.2.1.1 Component Overview

This software architecture diagram demonstrates the flow of the theater ticketing system. Showcasing how data is moved along through each step of the process.

The square blocks represent UI pages, movie catalog, time and theaters, login, etc. These will handle user input and display data.

Movie Catalog Page

- Displays a catalog of movies to scroll through
    - Each movie displays ratings and critic quotes.
- Pulls data from the movie database to know which movies to display
- Pulls data from critic review sites, such as Rotten Tomatoes.
- User inputs which movie they want to watch.

Time and Theaters Page

- Displays a list of showtimes and theater locations where the movie is shown.
- User selects time and location
- User selects whether they want deluxe or regular tickets

Seat Selection Page

- Displays a page of a top-down view of the theater seating arrangement.
- User selects the seats they want to reserve.

Payment Page

- Displays a payment ui, prompts the user to input their name and credit information.

Confirmation Page

- Displays receipt with ticket information and scannable code.
- Option to print ticket available.

Login Page

- Displays a prompt asking the user for email address and password.
- The button below displays a prompt to users if they don't already have an account to sign up.

Sign Up

- Display prompting the user for username, email, and password to create an account.

The cylinders represent databases, which are where the information is pulled from and stored, such as movies, theaters, and accounts.

List of Movies

- Stores movies and their showtimes, dates, and locations.

Theaters

- Stores physical theater layout and manages seat reservations.
    - Tracks which seats are available and unavailable.

User Accounts

- Stores user information, email, passwords, membership status, and order history.

The trapezoid represents a third-party service. In this case, it's a payment gateway that will handle transactions and critic sites.

Payment Gateway
- Third party api that handles transactions.

Critic Review Sites
- Provides critic reviews, quotes, and ratings

Connectors

Solid Arrows
- Represents the general path a user goes through when utilizing the system,
- Solid arrows also represent the information that's passed throughout each step.

Dashed Arrows
- Represents fetching data from the database.

4.2.1.2 Flow

The process begins at the movie catalog page, which fetches data, such as movies that are showing and their reviews, from critic review sites and the List of movies database, showcased via the dashed arrow connector.

When the user selects a movie, the user is navigated to a different page, which pulls the dates, time, and location of the movie from the database list of movies, shown by the dashed connector. On this page, the user makes a key decision that can alter the flow of the system. The user can choose regular or deluxe tickets, which determines which page they will be directed to next.

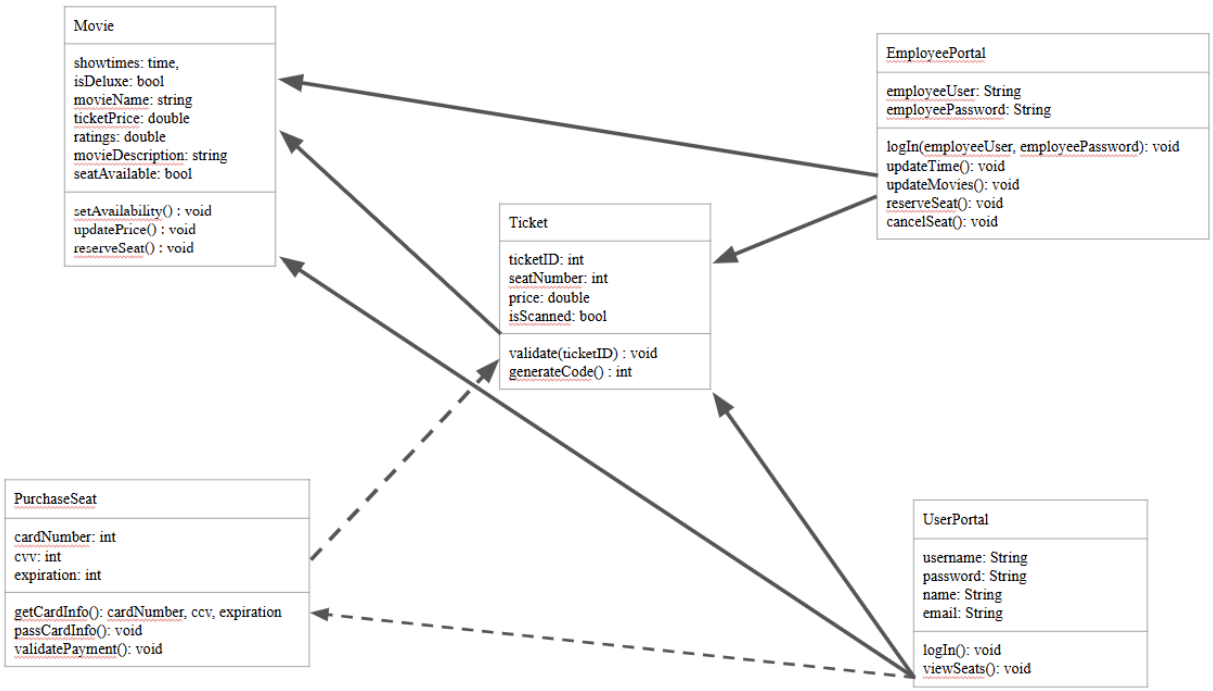
If the user selects deluxe tickets, they are directed to a seat selection page. This page pulls from the theater database, providing the theater's physical layout of seats and which ones are available or reserved. Once the user selects which seats they want to reserve, this information is sent back to the database to update that these seats are now reserved.

From either the Time and Theater's page or the Seat Selection page, the user is then navigated to the Payment page. The user inputs their credit information, and this info is then sent to a third-party payment gateway to handle the transaction, and whether the transaction was successful or not returned to the payment page.

If the purchase was successful, the user is directed to the confirmation page, which displays the ticket info and receipt of your purchase. This purchase is then sent to the account's database to update order history, if logged in.

The Login Page can be accessed from the movie catalog page, the homepage, or the payment page. This page pulls from the user accounts database, which keeps track of user name, email, password, order history, and membership status.

4.3 UML Diagram



4.3.1 UML Description

Class Descriptions

Movie

Stores data about movies:
Price, ratings, whether it's deluxe or not, showtimes, movie names/descriptions, and availability.

Ticket

Stores ticket information:
Ticket ID number, whether it was scanned, ticket price, and seat number.

EmployeePortal

Allows employees to adjust reserved seats (canceling/reserving) and has getters/setters for movie data. Includes employee login page.

UserPortal

Allows users to login and view available seats.

PurchaseSeat

Passes card information (card number, cvv, expiration) to a third party handler.

Class Attributes

Movie

showtimes: time - movie showing time
isDeluxe: bool - delux status
movieName: string - movie name
ticketPrice: double - ticket price
ratings: double - pulls movie rating out of 5 stars
movieDescription: string - pulls movie description
seatAvailable: bool - seat availability status

Ticket

ticketID: int - ticket ID number
seatNumber: int - in-theater seat number
price: double - price in USD
isScanned: bool - ticket scanned status

EmployeePortal

employeeUser: String - employee username
employeePassword: String - employee password

UserPortal

username: String
password: String
name: String
email: String

PurchaseSeat

cardNumber: int - card number
cvv: int - card CVV
expiration: int - card expiration date

Class Operations

Movie

setAvailability() : void - sets movie availability
updatePrice() : void - updates ticket prices
reserveSeat() : void - reserves a seat (user)

Ticket

validate(ticketID) : void - checks against ticket ID database to validate ticket ID
generateCode() : int - generates code for new ticket and adds to database

EmployeePortal

logIn(employeeUser, employeePassword): void - checks database and validates employee login
updateTime(): void - updates showtimes
updateMovies(): void - updates available movies
reserveSeat(): void - reserves a seat (employee - can override user seat reservation)

cancelSeat(): void - cancels seat reservation

UserPortal

logIn(): void - checks database and validates user login

viewSeats(): void - allows users to view available seats

PurchaseSeat

getCardInfo(): cardNumber, ccv, expiration - uses scanner to get card info at time of payment

passCardInfo(): void - passes card info to a third party handler to process it

validatePayment(): void - receives confirmation from a third party handler to validate ticket payment and release ticket

5. Development Plan and Timeline

5.1 Development Plan

Project split into parts so each member can work on different parts simultaneously.

Database: Creating the database for the movies, user accounts, and theater layouts.

Backend: Writing logic for the movement of data, connecting databases, connection to the payment gateway.

Frontend: Building UI for website and kiosk. Movie catalog homepage UI and seat selection UI.

Hardware Integration: Code that sends data to the thermal printer, code that reads data from a card reader.

5.2 Timeline

The client gave 4 month deadline for a viable prototype to user test
4 Phase schedule to ensure deadlines are met

Phase 1 (Month 1): Design Phase, UML diagram, and entity relationship diagram

Phase 2 (Month 2): Implementation, coding classes, and building UI

Phase 3 (Month 3): Testing and Debugging, testing idle kiosk, payment errors, and other edge cases.

Phase 4 (Month 4): Push to GitHub and present prototype to client.

5.3 Member Responsibilities

Member 1: Creating frontend UI for website and kiosk.

Member 2: Focus on backend development, writing logic for data movement, connection to critic sites, and payment gateway connection.

Member 3: Creating a database for movies, user accounts, and theater layouts.

Member 4: Focus on hardware integration, code for thermal printer and card reader.

6. Testing

6.1 Test Plan

This section describes the different levels used to verify the Theater Ticketing System.

6.1.1 Unit Level Testing

This level focuses on verifying the smallest testable parts of the application. These are the methods and functions of classes and objects, such as generateCode() and updateMovies().

An example test of the generateCode() function can be seen using test vectors of generating multiple ticket codes. This test ensures the function correctly generates a unique ticket code. The test would fail if a duplicate ticket code is found.

6.1.2 Functional Level Testing

This level of verification focuses on how the system's components interact. This level covers the connection to databases and third-party APIs, such as payment gateways and review sites.

An example of a test at the functional level would be selecting a seat. The Theater Database will use a test vector, “Seat A1,” to test if reserving this seat will change the available status to reserved within the database. This test would fail if reserving a seat failed to change the status of the seat within the database.

6.1.3 System Level Testing

This level of verification focuses on the entire system, testing a fully integrated version to evaluate its design and behavior and determine whether it meets customer standards. This test can mirror the entire ticket-purchasing journey, from the movie catalog page to the confirmation page.

An example of a test at the System level could be the entire flow of purchasing a ticket. The test vectors at this level would be searching for a movie, like “Inside Out,” going through the payment process using a payment gateway, and confirming the order to receive tickets and a receipt through email. This test would fail if any point during the process of purchasing a ticket didn’t work as expected. Such as a payment error, or if the tickets weren’t received.

6.2 Test Cases

This section provides 10 specific test case samples. These test cases are used to verify that the software meets specific requirements at the Unit, Functional, and System testing levels. Each test case contains a description of what feature is being tested and a specific set of test vectors, the test steps, to ensure that the test meets the expected output.

Github repository: https://github.com/JamjellyCT/Theater-Ticketing-System

A. Appendices

Appendices may be used to provide additional (and hopefully helpful) information. If present, the SRS should explicitly state whether the information contained within an appendix is to be considered as a part of the SRS’s overall set of requirements.

Example Appendices could include (initial) conceptual documents for the software project, marketing materials, minutes of meetings with the customer(s), etc.

A.1 Appendix 1

A.2 Appendix 2